

Theresia Mulyadi 陳友平 (112006221)

How the Code Works

This project implements preemptive multithreading with semaphore-based synchronization on the 8051 microcontroller. Unlike cooperative multithreading, the Producer and Consumer do not need to call `ThreadYield()` to switch threads. Instead, Timer 0 periodically interrupts the running thread and calls `myTimer0Handler()`, which saves the current thread state, selects the next runnable thread, restores its context, and returns using `RETI`.

The threading package stores each thread's saved stack pointer in internal RAM and uses a bitmap to track which thread IDs are active. Each thread has its own stack region and register bank. `Bootstrap()` initializes the thread system, creates `main()` as the first thread, enables the Timer 0 interrupt, starts Timer 0, and restores the context of the first thread. After that, `main()` initializes the bounded-buffer variables and semaphores, creates the Producer thread, and then runs the Consumer in thread 0.

This checkpoint replaces the previous single-buffer test case with a 3-deep bounded buffer. The buffer is stored in three consecutive memory locations, and the program uses head and tail variables to manage it as a circular queue. The Producer writes new characters into the position indicated by tail, then advances tail. The Consumer reads characters from the position indicated by head, then advances head. When either index reaches the buffer size of 3, it wraps back to 0.

Three semaphores are used to synchronize access to the bounded buffer. The mutex semaphore is initialized to 1 and ensures that only one thread accesses the shared buffer at a time. The empty semaphore is initialized to 3 because the buffer starts with three empty slots. The full semaphore is initialized to 0 because the buffer initially contains no characters.

The Producer generates characters from A to Z repeatedly. Before inserting a character, it calls `SemaphoreWait(empty)` to make sure there is at least one empty slot, then calls `SemaphoreWait(mutex)` to enter the critical section. Inside the critical section, it writes the character into the buffer and updates the tail index. After leaving the critical section, it calls `SemaphoreSignal(mutex)` to release the buffer and `SemaphoreSignal(full)` to indicate that one more filled slot is available.

The Consumer removes one character at a time from the buffer. It first calls `SemaphoreWait(full)` to make sure there is at least one available character, then calls `SemaphoreWait(mutex)` to enter the critical section. Inside the critical section, it reads the character from the buffer and updates the head index. After that, it calls `SemaphoreSignal(mutex)` to release the buffer and `SemaphoreSignal(empty)` to indicate that one more empty slot is available. The consumed character is then sent to the UART through SBUF.

The semaphore values show the state of the bounded buffer. When the Producer inserts an item, empty decreases and full increases. When the Consumer removes an item, full decreases and empty increases. For example, if the buffer is full, empty becomes 0 and full becomes 3. This confirms that all three buffer slots contain produced characters. The UART output continues printing characters in alphabetical order, showing that the Producer, Consumer, Timer 0 preemption, and semaphore synchronization are working correctly.

3.1 Screenshots for compilation

```
PS C:\Users\Theresia\Downloads\112006221_ppc3> make clean
powershell -NoProfile -Command "Remove-Item -ErrorAction SilentlyContinue *.hex,*.ihx,*.lnk,*.lst,*.map,*.mem,*.rel,*.rst,*.sym,*.asm,*.lk"
PS C:\Users\Theresia\Downloads\112006221_ppc3> make
sdcc -c testpreempt.c
sdcc -c preemptive.c
sdcc -o testpreempt.hex testpreempt.rel preemptive.rel
PS C:\Users\Theresia\Downloads\112006221_ppc3> Get-ChildItem

Directory: C:\Users\Theresia\Downloads\112006221_ppc3

Mode                LastWriteTime         Length Name
----                -
-----
-a-----         6/13/2026   6:25 AM             559 Makefile
-a-----         6/13/2026   7:54 AM          14639 preemptive.asm
-a-----         6/13/2026   6:58 AM           7396 preemptive.c
-a-----         6/13/2026   7:50 AM           2170 preemptive.h
-a-----         6/13/2026   7:54 AM          40145 preemptive.lst
-a-----         6/13/2026   7:54 AM           6265 preemptive.rel
-a-----         6/13/2026   7:54 AM          40799 preemptive.rst
-a-----         6/13/2026   7:54 AM          40876 preemptive.sym
-a-----         6/13/2026   7:54 AM          15673 testpreempt.asm
-a-----         6/13/2026   7:53 AM           4902 testpreempt.c
-a-----         6/13/2026   7:54 AM           2004 testpreempt.hex
-a-----         6/13/2026   7:54 AM            261 testpreempt.lk
-a-----         6/13/2026   7:54 AM          41062 testpreempt.lst
-a-----         6/13/2026   7:54 AM          17656 testpreempt.map
-a-----         6/13/2026   7:54 AM           1192 testpreempt.mem
-a-----         6/13/2026   7:54 AM          6553 testpreempt.rel
-a-----         6/13/2026   7:54 AM          41713 testpreempt.rst
-a-----         6/13/2026   7:54 AM          40696 testpreempt.sym

PS C:\Users\Theresia\Downloads\112006221_ppc3>
```

3.2

Part 1: When the Producer is running

The screenshot displays the Proteus IDE interface during a simulation. The top window shows the assembly code for the 'Producer' function, with the instruction 'INC 39H' highlighted. The register window shows the Program Counter (PC) at address 0x00D9, which is labeled '8051'. The memory window shows the contents of memory locations 0x00 to 0x7F. The bottom window shows the hardware simulation interface, including a keypad, a display, and various control buttons.

Assembly Code:

```
00B0 | SJMP 0E9H
00B2 | MOV A, 34H
00B4 | ADD A, #30H
00B6 | MOV R0, A
00B7 | MOV @R0, 35H
00B9 | MOV A, 34H
00BB | INC A
00BC | MOV 34H, A
00BE | MOV A, #03H
00C0 | CJNE A, 34H, 03H
00C3 | MOV 34H, #00H
00C6 | MOV A, #5AH
00C8 | CJNE A, 35H, 05H
00CB | MOV 35H, #41H
00CE | SJMP 05H
00D0 | MOV A, 35H
00D2 | INC A
00D3 | MOV 35H, A
00D5 | INC 37H
00D7* | INC 39H
00D9 | SJMP 0A9H
```

Register Window:

R/O	W/O	TH0	TL0	R7	B
0x00	0x00	0x03	0x07	0x00	0x00
R0	R1	TMOD	0x20	R6	ACC
0x00	0x00	0x00	0x00	0x00	0x42
R2	R3	TCN	0xD0	R5	PSW
0x00	0x00	0x00	0x00	0x00	0x08
R4	R5	PC	0x00D9	R4	IP
0x00	0x00	0x00	0x00	0x00	0x00
R6	R7	PSW	0x00	R3	IE
0x00	0x00	0x00	0x00	0x00	0x82
R8	R9	PCON	0x00	R2	DPH
0x00	0x00	0x00	0x00	0x00	0x00
R10	R11	DPL	0x00	R1	DPL
0x00	0x00	0x00	0x00	0x00	0x00
R12	R13	SP	0x4F	R0	SP
0x00	0x00	0x00	0x00	0x00	0x4F

Hardware Simulation Interface:

- Keypad: 12x3 grid with buttons labeled 1-9, *, #, and function keys.
- Display: 8-digit 7-segment display showing '8.8.8.8'.
- ADC: Analog-to-Digital Converter with a slider and a digital readout showing '11111111'.
- Motor: Motor control interface with a 'MAX' button and a 'Motor Enabled' indicator.
- UART: 8-bit UART @ 4800 Baud with 'Rx' and 'Tx' fields.
- Scope: Oscilloscope showing a signal waveform.

The Producer is running because the program counter is inside the Producer() function and currentThread at address 0x24 is 0x01, meaning thread 1 is active. The screenshot was taken after the Producer executed SemaphoreSignal(full), shown by the instruction INC 39H. The semaphore values are stored at known addresses: mutex at 0x37, empty at 0x38, and full at 0x39. After producing one item, empty becomes 0x02, showing that one empty slot was used, and full becomes 0x01, showing that one buffer slot now contains data. The mutex value is 0x01, meaning the Producer has released the critical section.

Part 2: When the Consumer is running

The screenshot shows the ED SIM5 IDE interface. The top panel displays the state of the Consumer thread, which is running. The registers section shows the Program Counter (PC) at address 0x012C, with a value of 8051. The assembly code window shows the instruction `INC 38H` at address 0x12C. The bottom panel shows a virtual hardware interface with a keyboard, a display showing '8888', and an ADC output of 11111111.

Registers:

R/O	W/O	TH0	TL0	R7	B
0x00	0x00	0x03	0x19	0x10	0x00
RXD	TXD	TMOD	0x20	R6	ACC
1	1	0x00	0x00	0x00	0x03
SCON	0x50	TCON	0xD0	R5	PSW
				0x00	0x00
				R4	IP
				0x00	0x00
				R3	IE
				0x00	0x82
				R2	PCON
				0x00	0x00
				R1	DPH
				0x30	0x00
				R0	DPL
				0x21	0x01
					SP
					0x3F

Memory:

addr	0x00	0x00	value
00	21	30	00
01	00	00	00
02	00	00	00
03	00	00	00
04	00	00	00
05	00	00	00
06	00	00	00
07	00	00	00
08	00	00	00
09	00	00	00
0A	00	00	00
0B	00	00	00
0C	00	00	00
0D	00	00	00
0E	00	00	00
0F	00	00	00
10	00	00	00
11	00	00	00
12	00	00	00
13	00	00	00
14	00	00	00
15	00	00	00
16	00	00	00
17	00	00	00
18	00	00	00
19	00	00	00
1A	00	00	00
1B	00	00	00
1C	00	00	00
1D	00	00	00
1E	00	00	00
1F	00	00	00
20	47	57	00
21	00	00	00
22	00	00	00
23	00	00	00
24	00	00	00
25	00	00	00
26	00	00	00
27	00	00	00
28	00	00	00
29	00	00	00
2A	00	00	00
2B	00	00	00
2C	00	00	00
2D	00	00	00
2E	00	00	00
2F	00	00	00
30	41	42	43
31	01	00	44
32	41	01	00
33	02	00	00
34	00	00	00
35	00	00	00
36	00	00	00
37	00	00	00
38	00	00	00
39	00	00	00
3A	00	00	00
3B	00	00	00
3C	00	00	00
3D	00	00	00
3E	00	00	00
3F	00	00	00

Assembly Code:

```

0110 JUMP 04H
0112 SETB 0AFH
0114 JUMP 0E9H
0116 MOV A, 33H
0118 ADD A, #30H
011A MOV R1, A
011B MOV 36H, @R1
011D MOV A, 33H
011F INC A
0120 MOV 33H, A
0122 MOV A, #03H
0124 CJNE A, 33H, 03H
0127 MOV 33H, #00H
012A INC 37H
012C* INC 38H
012E MOV 99H, 36H
0131 JBC 99H, 0B4H
0134 SJMP 0FBH
0136 MOV 33H, #00H
0139 MOV 34H, #00H
013C MOV 35H, #41H
    
```

Hardware Interface:

- DI: 1, LD: 1
- AND Gate Disabled
- Key Bounce Disabled
- Standard
- 0.0 V output
- Scope
- DAC
- BF: 0, AC: 0x00, IR: 0x00, DR: 0x00
- 8-bit UART @ 4800 Baud
- No Parity
- Rx Reset
- Tx Send
- 0.0 V input
- ADC
- Motor Enabled

The Consumer is running because the program counter is inside the Consumer() function and currentThread at address 0x24 is 0x00, meaning thread 0 is active. The screenshot is taken at the instruction `INC 38H`, which corresponds to SemaphoreSignal(empty). The semaphores are stored at known addresses: mutex at 0x37, empty at 0x38, and full at 0x39. After the Consumer removes one character from the buffer, empty increases to 0x01, showing that one buffer slot is now available, while full is 0x02, showing that two filled slots remain. The mutex value is 0x01, meaning the Consumer has released the critical section. The consumed character is stored at 0x36, which contains 0x43, representing 'C'.